

2025 - 2026

THE INTELLIGENT
ROGUELIKE :
ROGUIA

NOTRE ÉQUIPE

L'ÉQUIPE IA :



**Raykesh
RAVIROOBAN**

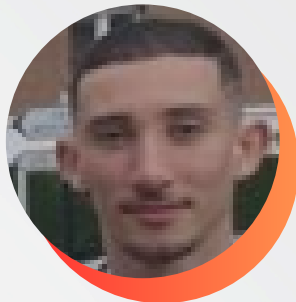


**Maxence
KLEIN**



**Coumba
MBAYE**

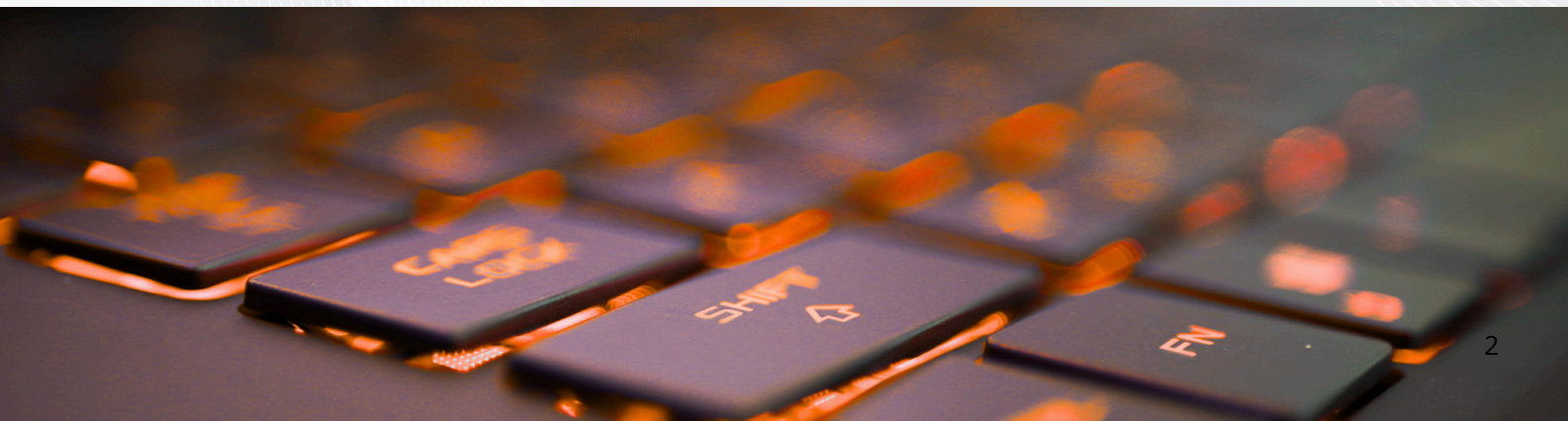
L'ÉQUIPE DATA :



**Sabri
MESSAOUDI**

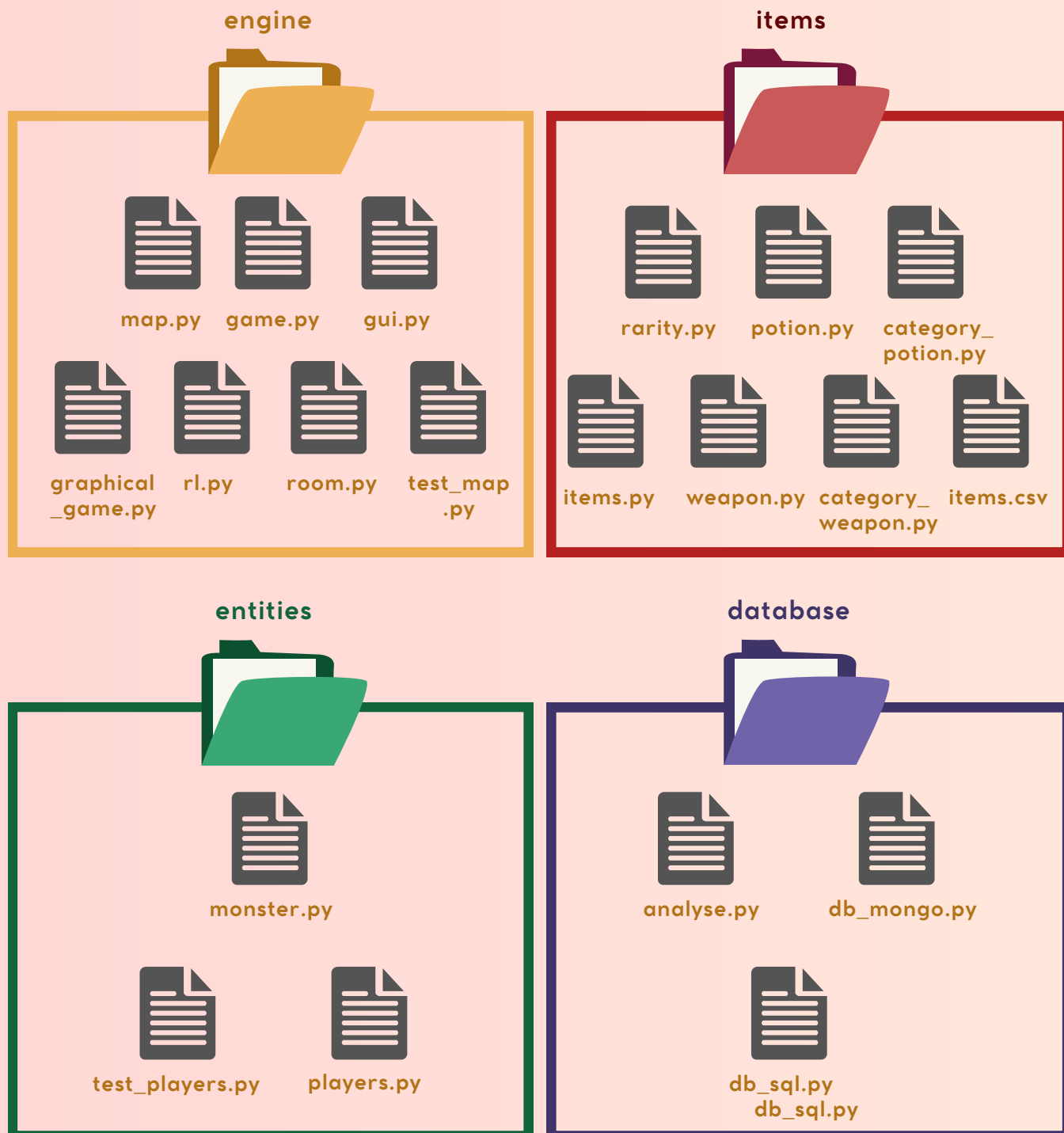


**Chrisphen
RINGUET**



Architecture de notre projet

Roguia a été développé autour de 4 grands dossiers :



SOMMAIRE

⊙1
PRINCIPE DU JEU

⊙2
PARTIE INTELLIGENCE ARTIFICIELLE

⊙3
GÉNÉRATION PROCÉDURALE

⊙4
LOGIQUE DE DIFFICULTÉ DU JEU

⊙5
PRÉ-ENTRAÎNEMENT

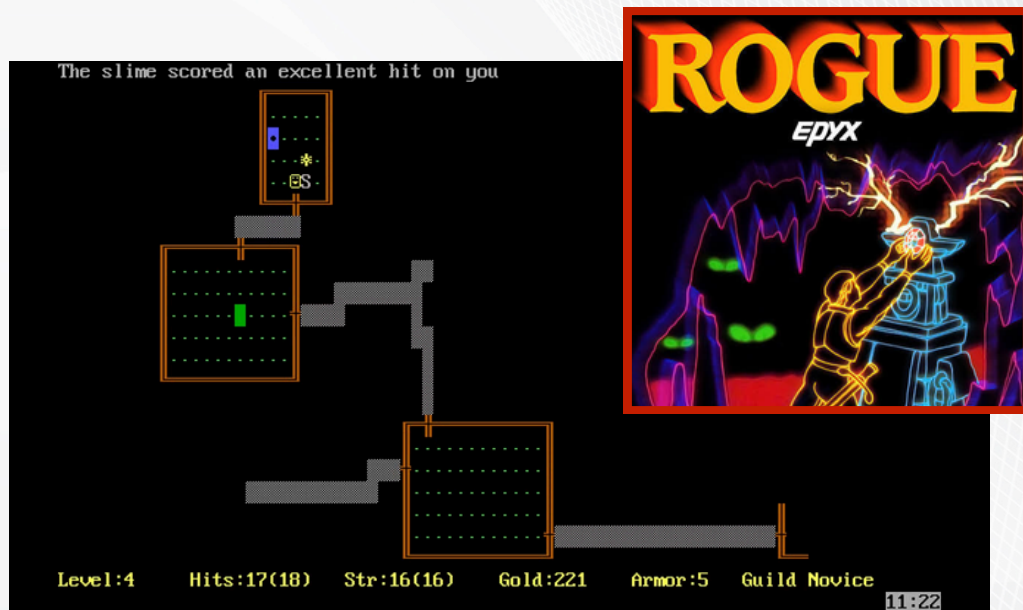
⊙6
PARTIE BASE DE DONNÉES

⊙7
TESTS

⊙8
OUVERTURE

PRINCIPE DU JEU

Le projet **Roguia** consiste à développer un **jeu vidéo 2D de type Roguelike**, principalement programmé en **Python**.



Capture d'écran du jeu original Rogue (1980)

Les **Roguelikes** (littéralement: "inspiré de *Rogue*") sont des jeux basés sur les codes de *Rogue*, un jeu sortie en 1980. Le but du joueur était de **finir le plus de donjons** (suite de salles) sans se faire tuer par les monstres qui infestent le domaine. Il peut, à sa guise, récupérer des **objets** (armes, potions,...) pour le guider dans sa quête. Plus il tue de monstre et fini de salle, plus il **augmente son score** : la partie se termine lorsque le joueur perd son dernier **point de vie**.

Ce qu'on créer en + :

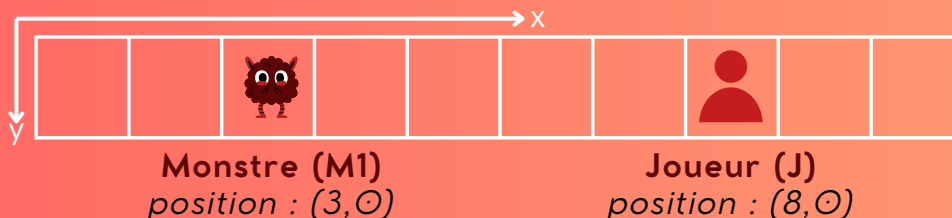
Nous avons intégré une **intelligence artificielle évolutive** aux monstres, leur permettant de **s'adapter dynamiquement** aux stratégies et au niveau de chaque joueur dès le début de la partie. Cette mécanique garantit une **expérience de jeu immersive, renouvelée** et pleinement **personnalisée**.

PARTIE IA

Le **Q Learning** est un algorithme d'**Apprentissage par Renforcement** centrale dans notre projet. C'est le processus du machine learning qui permet à nos **monstres d'apprendre à prendre les meilleures décisions** pour se diriger vers le joueur et l'attaquer. Il est basé sur X concepts détaillés ci-dessous.

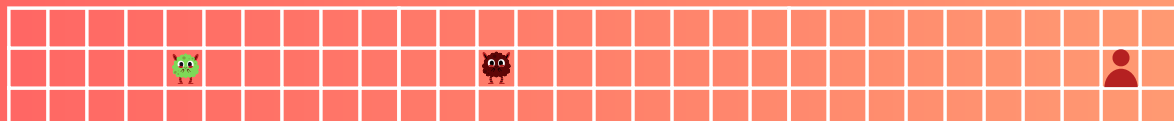
L'état (s) :

Chaque monstre est représenté par un couple d'entier (x,y) représentant sa distance relative au joueur : c'est son **état s**.



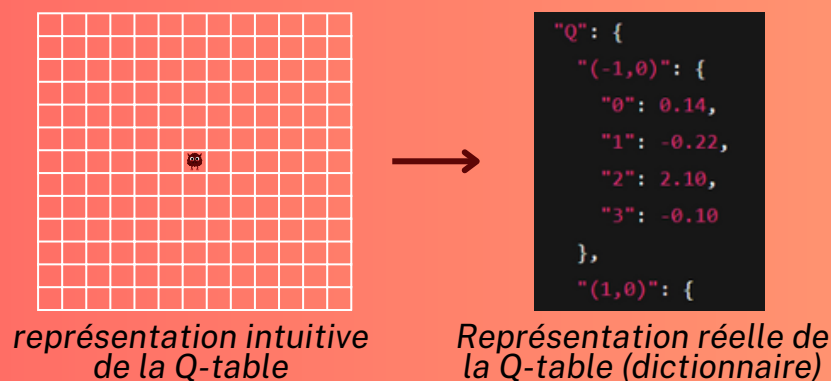
Ici, la position du monstre est de (3,0) et celle du joueur de (8,0). La distance entre eux, et donc l'**état** du monstre, est de **s = (5,0)**.

Les deux paramètres x et y de l'état sont compris entre -6 et 6 pour limiter la taille de la **Q-table**, qui contient l'ensemble des valeurs pour tous les états des monstres :



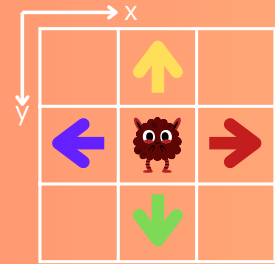
état de **M2** : (6, 0) état de **M1** : (6, 0)

Ainsi, même si les deux monstres M1 et M2 ne sont pas à la même position, ils ont le **même état** et apprendront à agir de la même manière : aller à **droite** en direction du joueur. Grâce à cette solution, notre Q-table ne contiendra plus que **169 cases** au lieu d'avoir autant de cases que la map.



Les actions (a) :

Les monstres de notre jeu ne peuvent effectuer que **4 actions** possible : se déplacer vers le **haut**, le **bas**, à **gauche** et à **droite**. A chaque unité de temps, le monstre **calculera son état**, puis à l'aide de la **stratégie Epsilon-greedy**, il choisira quelle action effectuer puis sera **récompensé** positivement ou négativement en fonction du résultat de son action.



Epsilon-greedy

Pour sélectionner la meilleure action à faire pour chaque changement d'état du monstre, on utilise un algorithme dit Epsilon-greedy.

```
def select(self, s):  
    if random.random() < self.epsilon:  
        return random.choice(self.actions)  
    qs = self.Q[s]  
    best_a, best_q = None, float("-inf")  
    for a in self.actions:  
        q = qs.get(a, 0.0)  
        if q > best_q:  
            best_q, best_a = q, a  
    return best_a if best_a is not None else random.choice(self.actions)
```

Exploration

Dictionnaire valeurs des états

Recherche de la
meilleure valeur

A chaque nouvel état, le monstre tire un nombre aléatoire entre 0 et 1. Si celui-ci est inférieur à un epsilon donné, alors le monstre choisit une des actions au hasard. Sinon, il fouille dans sa Q-table dans laquelle il stocke les valeurs de chaque action par état, pour choisir la meilleure.

Toutes les 10 unités de temps, on décrémente légèrement epsilon pour simuler une "prise de confiance" de la part du monstre.

Les récompenses (r) :

En fonction du nouvel état du monstre, l'algorithme donne une récompense au monstre sous la forme d'un entier naturel. Toutes les récompenses possibles sont regroupées dans ce tableau :

Chaque pas	-1
Mur	-5
Collision autre monstre	-2
S'approche du joueur	+0.5*différence de distance
S'éloigne du joueur	-0.5*différence de distance
Atteint le joueur	+ 20

ex : Le monstre se déplace à droite (-1), se heurte contre un mur (-5) mais est plus prêt du joueur (+0,5 * 1) => -5,5 de récompense



Boucle de fonctionnement :

Tout se passe à l'intérieur de la classe `Monster` via la méthode `rl_step(game_map, player_position)`.

Monster
<code>die(drop_rate)</code>
<code>get_damage()</code>
<code>get_equiped_item()</code>
<code>get_is_alive()</code>
<code>get_position()</code>
<code>get_pv()</code>
<code>get_x()</code>
<code>get_y()</code>
<code>heal(soin)</code>
<code>kill(drop_rate)</code>
<code>kill all monsters()</code>
<code>rl_step(game_map, player_pos)</code>
<code>set_equiped_item(equiped_item)</code>
<code>set_position(x, y)</code>
<code>set_pv(pv)</code>
<code>set_speed(speed: float)</code>
<code>set_x(x)</code>
<code>set_y(y)</code>

```
self._ensure_rl()
s = self._state(player_pos)
a = self.rl_agent.select(s)
s2, r, done, _ = self._try_action_on_map(game_map,
                                         a,
                                         player_pos)
self.rl_agent.update(s, a, r, s2, done)
```

Elle est appelée à chaque **tick** et agit ainsi :

1. Vérifie que le système RL est actif;
2. Calcule la **position relative** du monstre au joueur
3. Sélectionne l'**action** avec epsilon-greedy
4. Applique l'action sur la map et en tire les **récompenses** et le **nouvel état**
5. Met à jour la Q-table

Mise à jour de la Q-table :

Une fois la récompense obtenue, le monstre connaît la **valeur** qu'a eu son action sur la map à son état donné et est donc capable de l'inscrire dans sa Q-table, que l'on peut naïvement considérer comme la **mémoire** du monstre.

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

$Q(s, a)$: valeur de l'action a dans l'état s

R : récompense immédiate

α : taux d'apprentissage

γ : facteur d'actualisation

s' : état suivant

C'est avec cette formule que le monstre **modifie la valeur de l'action** à l'état associé dans sa table.

Après des milliers d'actions, il obtient une **table de valeurs** fournie et affûtée qui lui permettra de **choisir les actions** qui le **mèneront vers le joueur**.

Ce système complet d'apprentissage par renforcement permettra au monstre d'apprendre le meilleur comportement à adopter en fonction du joueur. Il ne lui reste plus qu'à s'entraîner.

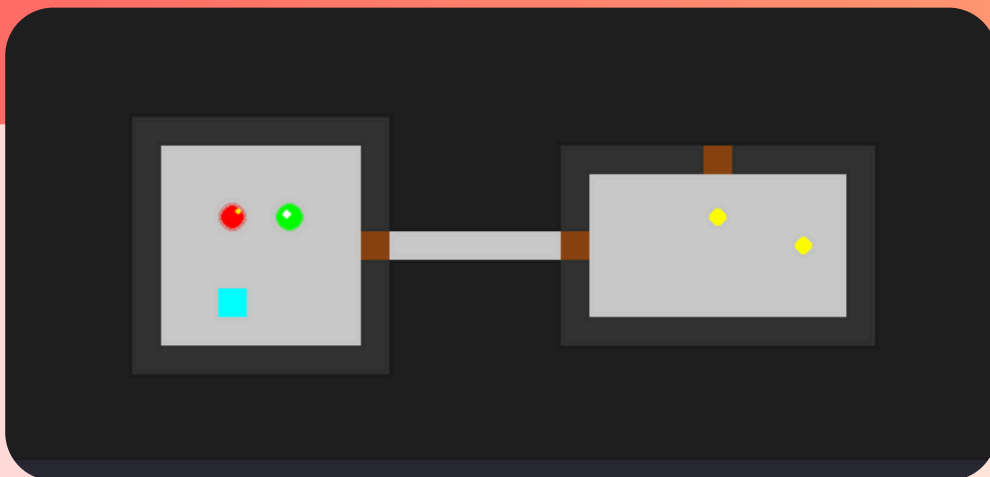


GÉNÉRATION PROCÉDURALE :

La **génération procédurale** est le procédé de notre projet qui permet de **construire des donjons uniques** composés d'un ensemble de salles. Chaque donjon créé est différent, car à chaque génération de salle, la taille ainsi que la disposition des portes varient. L'un des objectifs techniques majeurs est d'assurer la **solvabilité** d'un niveau : c'est-à-dire qu'il doit obligatoirement exister un chemin praticable entre le point d'apparition du joueur et la sortie.

La première étape consiste à **placer les salles sur la grille** de jeu. Nous utilisons une méthode d'essai-erreur. L'algorithme tente de poser une salle rectangulaire en tirant **aléatoirement ses dimensions** (largeur et hauteur) ainsi que sa position sur la carte.

Avant de valider la création de cette salle, le système vérifie qu'elle ne chevauche aucune autre salle déjà présente. Nous appliquons également une **marge de sécurité** autour de chaque salle afin d'éviter qu'elles ne soient collées les unes aux autres. Si la salle proposée entre en contact avec une zone occupée, elle est rejetée et l'algorithme recommence avec de nouvelles coordonnées, jusqu'à atteindre le nombre de salles souhaité.



Ci-dessus, nous présentons un exemple de carte entièrement générée de manière aléatoire, sur laquelle des monstres spawn (apparaissent) à des positions également aléatoires.

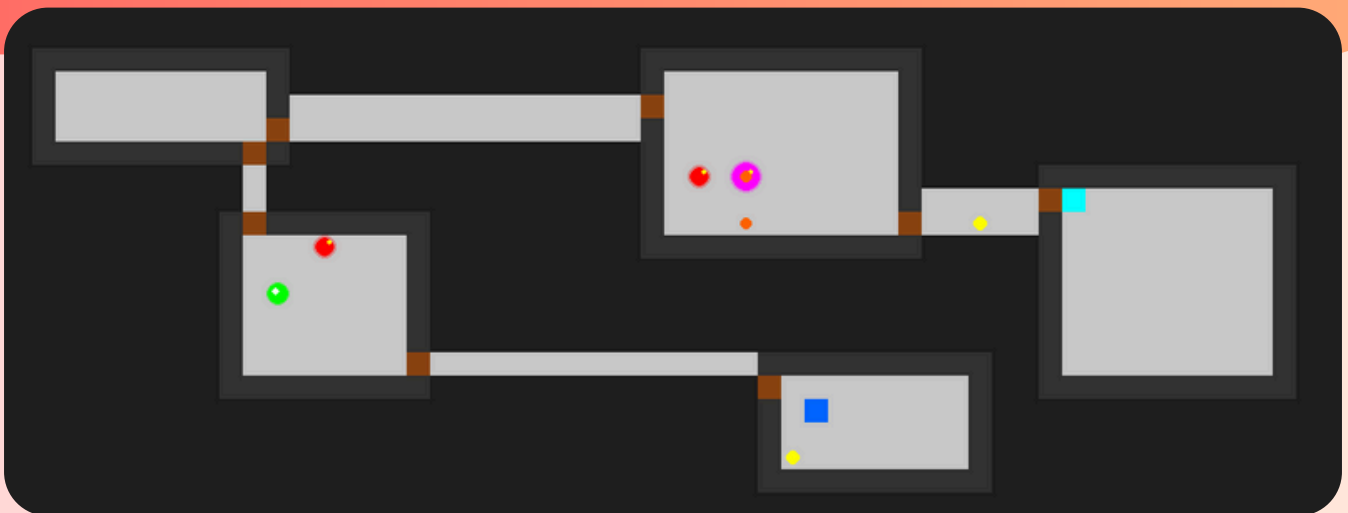


Une fois que les salles sont **positionnées**, on les relie afin de former le **donjon**. Pour garantir que l'ensemble de la carte est accessible, nous utilisons une structure de **connexion en chaîne simple** ce qui signifie que une salle 1 est reliée à la salle 2, la salle 2 à la salle 3... jusqu'à la dernière salle.

Pour tracer les couloirs, l'algorithme de la génération procédurale relie le centre de la salle de départ à celui de la salle d'arrivée. Il privilégie un tracé en forme de "L" (angle droit), également appelé **dog-leg**. Si ce chemin simple est bloqué par un obstacle imprévu, le système bascule vers une recherche de chemin plus complexe basée sur l'algorithme BFS (Breadth-First Search) afin de trouver un tracé alternatif.

Cette approche garantit mathématiquement qu'aucune salle ne reste isolée et que le donjon est toujours entièrement praticable.

Vu que la map change à chaque partie, le donjon n'est jamais le même. Cette variation est cruciale pour l'IA : elle empêche les monstres de simplement retenir la carte et de suivre un chemin tout tracé. À la place, l'agent doit observer ce qui l'entoure un couloir, un coin ou une grande salle et décider de ses mouvements en fonction de la situation, à chaque instant.



Ci-dessus, on voit un étage complet avec ses couloirs et ses cinq salles (le maximum par étage étant de cinq).



LOGIQUE DE DIFFICULTÉ DU JEU :

L'innovation majeure de ce projet réside dans le fait que la difficulté ne provient pas uniquement des points de vie, mais de la **compétence décisionnelle des ennemis**. Nous avons défini **trois phases** de difficulté cognitive :

Phase d'Initiation (Niveaux 1-2) : Les monstres utilisent le **modèle pré-entraîné** (Base Q-Table). Ils possèdent les fondamentaux (se déplacer, attaquer au contact) mais restent **génériques**. C'est la phase **d'échauffement** pour le joueur.

Phase d'Adaptation : Les monstres utilisent l'**IA Locale du joueur**. Grâce à l'apprentissage continu, le monstre commence à **s'adapter aux habitudes spécifiques** du joueur.

Phase de Challenge "Super Monstre" (Niveaux 5+) : L'apparition du **Super Monstre** introduit une **difficulté abrupte**. Cet agent utilise le **Modèle Global**, entraîné sur l'ensemble des données de la **communauté** (Big Data). Il représente le pic de **difficulté ultime**, capable de stratégies complexes apprises contre des centaines de joueurs différents.

La **chance P** qu'un "Super Monstre" apparaisse dans une salle est définie par :

$$P(\text{SuperMonstre}) = \min(1.0, \max(0.0, (d - 2) * 0.1))$$

Cette formule garantit le comportement suivant :

- **d < 3** : P=0 (Aucune chance, débutant).
- **d = 3** : P=0.1 (10% de chance, introduction rare).
- **d = 7** : P=0.5 (50% de chance, menace fréquente).
- **d > 12** : P=1.0 (Certitude, environnement hostile maximal).

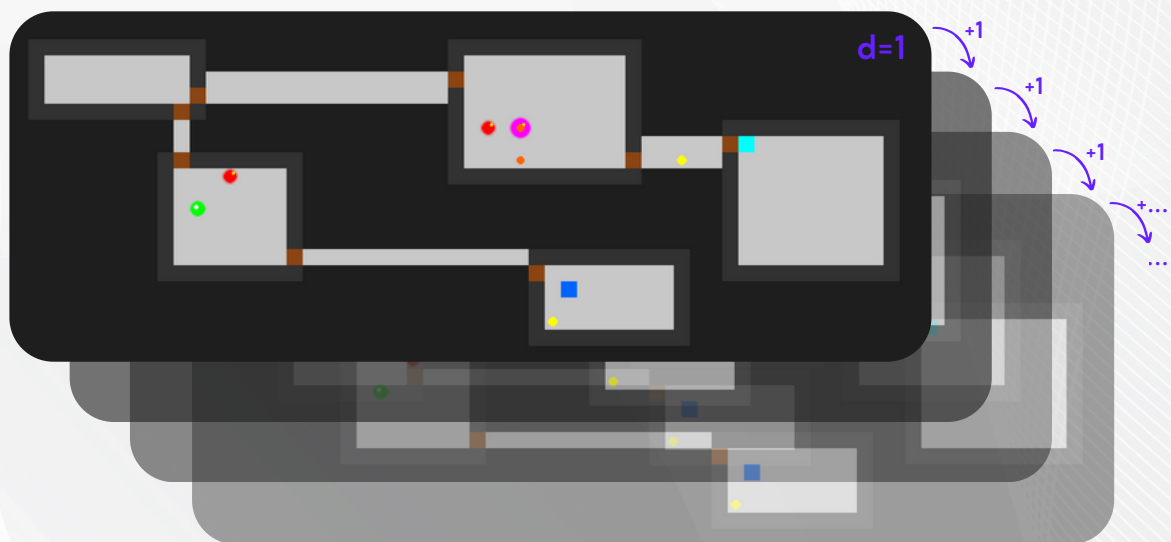


La gestion de la difficulté est un enjeu crucial dans un Roguelike. Si le jeu est trop facile, le joueur s'ennuie ; s'il est trop difficile, il est frustré. Notre système repose sur une difficulté hybride, pilotée par une variable centrale :

la **Profondeur** (Depth), notée **d**

Cette variable **s'incrémente ($d+1$)** à chaque fois que le joueur atteint la **porte finale d'un niveau**. Elle influence le jeu selon deux axes distincts :

- une progression **quantitative** (statistiques)
- et une progression **qualitative** (intelligence artificielle)



Nous appliquons un coefficient multiplicateur basé sur la **profondeur d** aux **attributs des ennemis** générés :

- **Points de Vie (PV)** : $PV_{monstre} = 50 + (d * 10)$.
 - Exemple : Un monstre au niveau 1 a 60 PV. Au niveau 10, il en a 150.
- **Densité de Population** : Le nombre d'ennemis par salle augmente selon la formule $[2 + d/2]$.
- **Équipement** : La probabilité qu'un monstre possède une arme de rareté supérieure (et donc des **dégâts plus élevés**) augmente avec la profondeur.



PRÉ-ENTRAÎNEMENT

Pour éviter que les monstres soient totalement **ignares** lorsqu'un joueur commence notre jeu, nous avons mis en place un **pré-entraînement** contre des **bots** (des agents automatisés) qui vont prendre le rôle de joueur afin de commencer à façonner leur **Q-table**.

Il y a 3 types de bots contre lesquels les monstres vont se battre :



- Le **bot agressif** va toujours se déplacer vers le monstre le plus proche et l'attaquer. Ce comportement va obliger le monstre à apprendre à gérer des confrontations directes et à optimiser l'esquive.



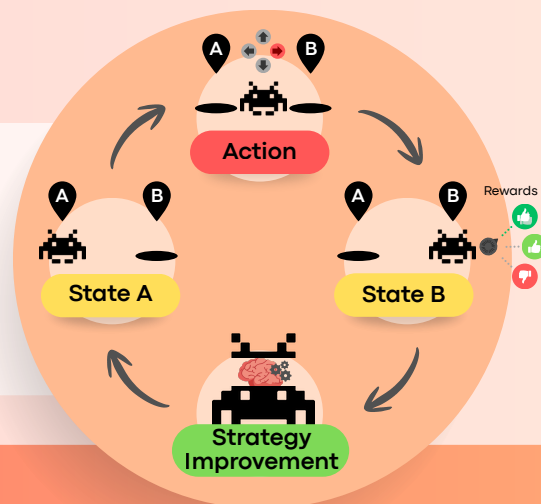
- Le **bot fuyard** quant à lui, va chercher à s'éloigner lorsqu'un ennemi approche. Face à lui, le monstre apprendra la poursuite du joueur.



- Le **bot aléatoire** choisit aléatoirement entre se comporter comme un bot agressif ou un bot fuyard.

L'implémentation de ces bots se fait dans la classe **Player**.

Le monstre adapte sa stratégie grâce au Q-Learning expliqué plus tôt. Son système est résumé sur ce schéma :



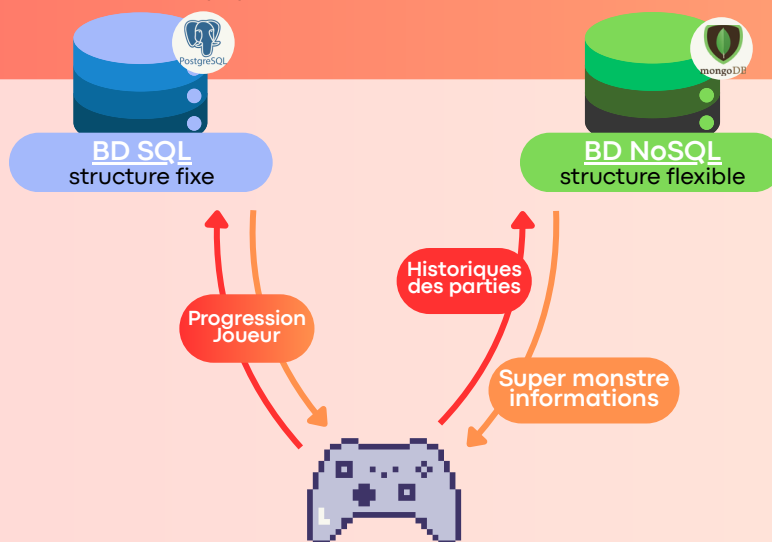
À l'issue de la phase de pré-entraînement, la Q-table est **sauvegardée localement** (base_q_table.pkl) afin d'être **réutilisée pour chaque nouveau joueur créé**. Cette intelligence des monstres constitue une base commune servant de point de départ à chaque nouvelle partie.

PARTIE DATA

La **partie data** concerne toute la partie des collectes des données de jeu (Items...) et données destinés au pré-entraînement.

L'objectif principal de était de concevoir un système capable de gérer deux flux de données distincts et c'est pour cela que Nous avons adopté une architecture hybride (**SQL et NoSQL**) pour répondre à ces deux besoins.

- Les données transactionnelles du jeu (état du joueur, inventaires) via une BDD relationnelle.
- Les données analytiques pour l'IA (logs d'entraînement) et l'analyse via une base NoSQL et un pipeline de traitement.



PostgreSQL :

La base **PostgreSQL** est le cœur transactionnel du jeu. Elle garantit la persistance, la cohérence et l'intégrité des données de jeu critiques.

- Schéma : Le schéma est défini dans database/init_db_sql.py. Il modélise les entités principales :
 - utilisateurs : **Authentification des comptes.**
 - joueurs : Statistiques du personnage (PV, XP, niveau) et chemin vers sa Q-Table.
 - items : Définition statique de tous les objets du jeu (armes, potions).
 - inventaires : Table de liaison (N-N) qui connecte les joueurs aux items avec une quantité.
 - parties et rooms : Permettent de suivre les sessions de jeu.



Interaction avec la base SQL :

L'interaction entre le **moteur de jeu** (engine/game.py) et la **base** est encapsulée dans le module **database/db_sql.py**. Ce module agit comme une couche d'abstraction qui expose des méthodes métier (ex: *sauvegarder_inventaire*, *update_joueur_stats*) et gère les connexions, garantissant que la **persistance est transparente pour le reste du code**.

Extrait de notre base SQL

Items :

id	nom	description	rarity	item_type	damage	defense	range	category	durability	potency	duration
integer	varchar(100)	text	varchar(50)	varchar	integer	integer	double precision	varchar(50)	integer	integer	integer
1	Epée émoussée	Une épée émoussée mais es	common	weapon	5	0	1	melee	90	0	0
2	Hache de guerre	Une hache lourde capable c	common	weapon	12	0	1	melee	80	0	0
3	Arc de chasseur de champi	Un arc léger et précis pour i	rare	weapon	8	0	5	distance	70	0	0
4	Gros bâton magique	Un bâton imprégné de p	epic	weapon	15	0	4	magic	60	0	0
5	Dague empoisonnée des té	Une petite lame recouverte	rare	weapon	7	0	1.5	melee	85	10	3
6	Bouclier en bois de boulot	Un bouclier simple mais util	common	weapon	0	5	0	shield	100	0	0
7	Potion de soin	Redonne progressivement c	common	potion	0	0	0	health	0	50	5
8	Potion de force	Augmente temporairement	common	potion	0	0	0	strenght	0	20	10

Inventaire :

id	joueur_id	item_id	quantite
int4	int4	int4	int4
200	2	1	3
201	2	15	2
202	2	2	2
203	2	9	2
204	2	17	1
205	2	7	2



Q-Tables via PostgreSQL :

Pour personnaliser la difficulté, chaque joueur doit posséder sa propre **mémoire** (Q-Table). Plutôt que de stocker ces données volumineuses directement dans la base de données, nous avons opté pour une **gestion par référence**.

PostgreSQL ne contient pas l'intelligence artificielle elle-même, mais agit comme un **annuaire**. La table joueurs contient une colonne `q_table_path` qui stocke uniquement le chemin vers un fichier binaire `.pkl` sur le serveur.

q_table_path selon joueur_id

Player :



id	ut...	i.	nom	niveau	xp	pv	q_table_path
2	2	→	Monster Slayer	7	500	105	models/qtables/joueur_2.pkl
3	3	→	RAYKESH	3	150	105	models/qtables/joueur_3.pkl
4	3	→	raykesh	7	275	35	models/qtables/joueur_4.pkl
5	4	→	monster hunter	2	0	85	models/qtables/joueur_5.pkl
6	5	→	Yoi	3	125	100	models/qtables/joueur_6.pkl

Fonctionnement :

1. **Connexion** : Au lancement (`run_game`), le jeu requête l'ID du joueur dans PostgreSQL pour récupérer le chemin du fichier.
2. **Chargement** : Le fichier **.pkl désigné** est chargé en mémoire vive via la librairie **pickle**. Si le fichier n'existe pas (nouveau joueur), le système copie le modèle **pré-entraîné** (`base_q_table.pkl`).
3. **Sauvegarde** : En fin de partie, la Q-Table mise à jour est **réécrite** dans le fichier local, sans alourdir la base de données.

Cette séparation (SQL pour les métadonnées, Système de Fichiers pour les modèles IA) optimise les **performances**. Elle évite de saturer la base de données avec des objets binaires lourds et garantit un chargement rapide de l'IA.



Base de Données non Relationnelle (MongoDB) :

Tandis que PostgreSQL gère les données critiques et structurées (comptes joueurs, inventaires), nous avons besoin d'une solution différente pour gérer le **volume massif de données générées par les combats**. C'est ici qu'intervient notre base **NoSQL, MongoDB**.

L'objectif de cette base est de capturer **chaque interaction** entre les joueurs et les monstres pour constituer une mémoire collective. Contrairement à une base SQL classique qui serait ralentie par des millions d'insertions de **logs**, MongoDB nous permet d'ingérer ces données à haute fréquence

Chaque **déplacement**, chaque **attaque** et chaque **erreur** du monstre est enregistré sous forme de **Document JSON**. C'est cette masse de données qui sert de matière première pour l'entraînement du "**Super Monstre**".

Exemple de log collecté pour l'entraînement du super monstre :

```
_id: ObjectId('690dce3384e8f9b319b301d8')
type: "transition"
ts: "2025-10-23T20:48:58.832523Z"
episode_id: "0aa6611c-71a7-43e2-8738-2ab3a243e826"
monster_id: 3046402594800
s: Array (2)
  0: -6
  1: -3
a: 0
r: -0.4
s2: Array (2)
  0: -6
  1: -2
done: false
tick: 85
pos: Array (2)
location: "room"
```



Le Pipeline d'Entraînement Global :

L'architecture NoSQL permet de découpler le jeu de l'analyse :

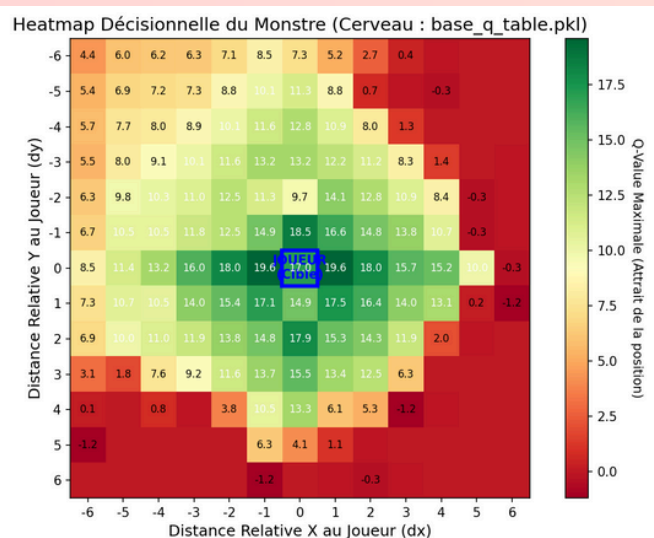
1. **Collecte (Runtime)** : Pendant que les utilisateurs jouent, le GameLogger envoie les logs vers MongoDB de manière asynchrone pour ne pas ralentir le jeu.
2. **Traitement (Offline)** : Périodiquement, le script `train_global_model.py` effectue une agrégation sur ces millions de documents. Il recalcule une Q-Table "moyenne" optimisée.
3. **Redistribution** : Ce nouveau modèle est sauvegardé (`global_q_table.pkl`) et sera chargé par les "Super Monstres" lors des prochaines parties

Le choix de **MongoDB** par rapport à SQL se justifie par trois critères :

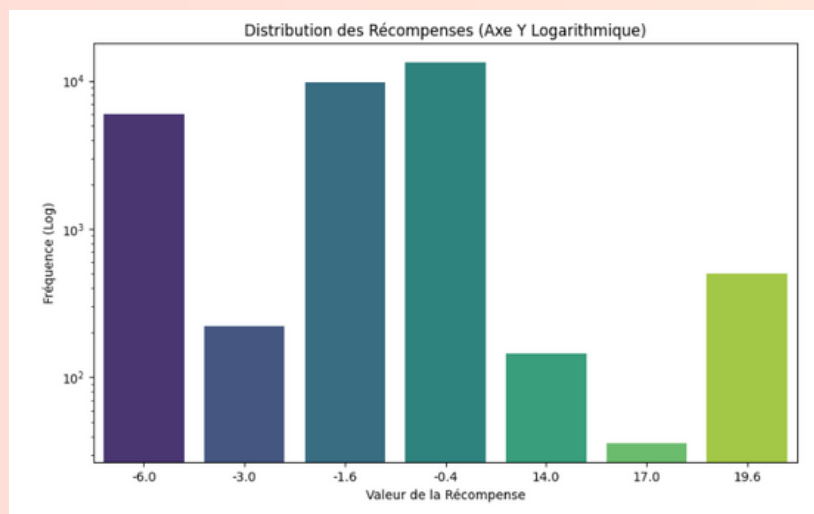
- **Volume** : Une seule partie de 10 minutes peut générer 2000 logs. Sur 100 joueurs, SQL saturerait rapidement.
- **Variété** : Le format JSON flexible nous permet d'ajouter de nouvelles métriques (ex: PV restants, arme utilisée) sans devoir modifier la structure de la base de données existante.
- **Vitesse** : La priorité est la vitesse d'écriture pour ne jamais impacter la fluidité du jeu.

L'analyse :

HeatMap d'un monstre de base :



Graphique de la Répartition des récompenses :



TESTS

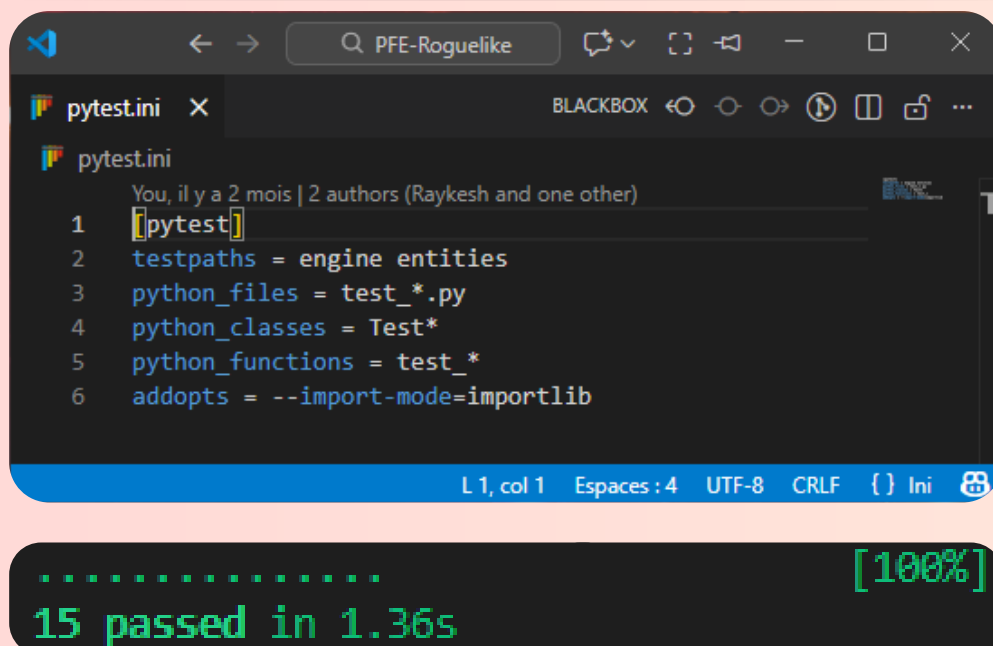
Pour assurer la fiabilité de notre projet, il a été important de faire des **tests** notamment pour éviter que le **pré-entraînement ne se base que sur des erreurs**.

Pour ce faire nous avons utilisé la bibliothèque **Pytest** qui permet d'automatiser les tests unitaires. Il permet de lancer tous les fichiers tests à l'aide d'un fichier script test **run_tests.py**.

Nous avons deux fichiers tests principaux :

test_player.py : pour tester les **méthodes** les plus importantes de la classe **Player** : Les déplacements du joueur, son inventaire et l'attaque (`_player_attack`) sur un monstre.

test_map.py : pour tester les **méthodes** les plus importantes de la classe **Map**. La gestion des projectiles est aussi testée pour savoir s'ils ont une portée cohérente, s'ils apparaissent. On teste si la carte est bien générée procéduralement avec les bons couloirs, la bonne taille etc.



The image shows a VS Code editor window with a file named `pytest.ini` open. The file contains the following configuration:

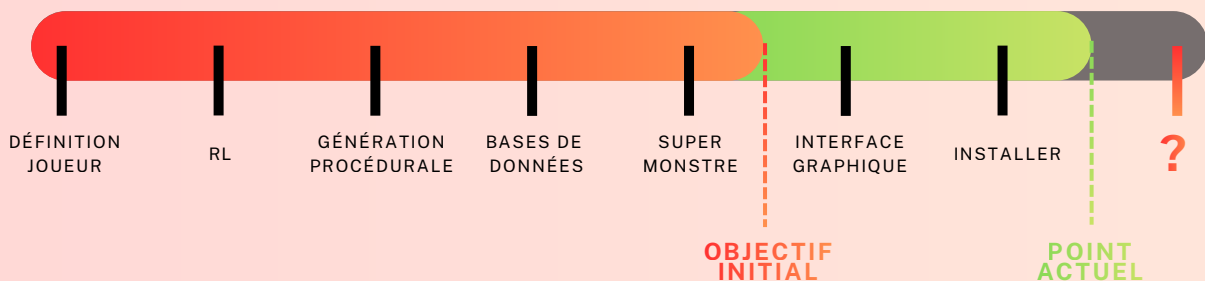
```
1 [pytest]
2 testpaths = engine entities
3 python_files = test_*.py
4 python_classes = Test*
5 python_functions = test_*
6 addopts = --import-mode=importlib
```

Below the editor, a terminal window displays the output of a pytest run:

```
..... [100%]
15 passed in 1.36s
```


OUVERTURE :

Comme le montre notre diagramme de progression ci-dessous, nous avons **atteint l'objectif** que nous nous étions fixé au début de notre projet de fin d'étude, il y a environ cinq mois. Pour améliorer **Roguia**, nous avons également mis en place une **interface graphique** ainsi qu'un **installateur** qui, à partir du fichier .env, permet d'installer le jeu et d'y jouer.



Améliorations Possibles

Les améliorations qui pourraient être envisagées dans le futur sont les suivantes :

- Permettre à chaque monstre d'avoir la **position des autres monstres**. Cela reviendrait à ajouter un paramètre en plus à l'état ce qui leur permettrait de développer des stratégies de **collaboration** plus avancées.
- Ajouter des éléments de décor et améliorer l'interface graphique afin de rendre l'expérience de jeu plus **immersive** et plus **user-friendly**.
- Introduire plusieurs **types de monstres** avec des comportements différents.



ANNEXE :

RÉFÉRENCES :

- [1] Watkins, C. J. C. H. (1989). Learning from Delayed Rewards. Ph.D. thesis, King's College, Cambridge.
- [2] Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning : An Introduction (2nd ed.). The MIT Press.
- [3] Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations : Theory and application to reward shaping. In Proceedings of the 16th International Conference on Machine Learning (ICML '99), pp. 278-287.
- [4] Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529-533.
- [5] Lange, S., Gabel, T., & Riedmiller, M. (2012). Batch Reinforcement Learning. In Reinforcement Learning : State-of-the-Art (pp. 45-73). Springer.
- [6] McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). Communication-efficient learning of deep networks from decentralized data. Artificial Intelligence and Statistics (AISTATS).
- [7] Shin, W. (2017). PyGame : A Primer on Game Programming in Python. Apress.
- [8] Chodorow, K. (2013). MongoDB : The Definitive Guide (2nd ed.). O'Reilly Media.
- [9] Yannakakis, G. N., & Togelius, J. (2018). Artificial Intelligence and Games. Springer. (Pour la partie Génération Procédurale et Roguelike).
- [10] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multiagent actor-critic for mixed cooperative-competitive environments. Advances in Neural Information Processing Systems (NIPS).
- [11] Vinyals, O., et al. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. Nature, 575(7782), 350-354.
- [12] McKinney, W. (2010). Data Structures for Statistical Computing in Python (Pandas). In Proceedings of the 9th Python in Science Conference (Vol. 445, pp. 51-56)
- GeeksforGeeks. "Q-Learning in Reinforcement Learning (Python)" : <https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/GeeksforGeeks>
- DataScientest. "Q-learning – Machine Learning with reinforcement learning" : <https://datascientest.com/en/q-learning-machine-learning-with-reinforcement-learning> DataScientest
- MachineLearningMastery. "A Gentle Introduction to Q-Learning" : <https://machinelearningmastery.com/a-gentle-introduction-to-q-learning/>
- Chat-gpt. Open-AI. (Brainstorming, reformulation, clarification, traductions préliminaires, vulgarisation scientifique.)

GITHUB :

<https://github.com/RaykeshR/PFE-Roguelike>

GITHUB README :

<https://github.com/RaykeshR/PFE-Roguelike/blob/Dev/README.md>

GITHUB PAGES :

<https://raykeshr.github.io/PFE-Roguelike/>

PRÉSENTATION DE MI-PARCOURS :

<https://www.canva.com/design/DAG3ZXifNJU/eVEuzn3wVc2nogXEW3u7Sg/edit>

DOCUMENTATION TECHNIQUE ET FORMALISATION MATHÉMATIQUE :

<https://raykeshr.github.io/PFE-Roguelike/docs/documentation.pdf>



ANNEXE :

A.1 Architecture Logicielle

Le projet est structuré selon une séparation claire des responsabilités (Separation of Concerns). Le Tableau 2 détaille les modules principaux.

Module / Dossier	Rôle et Responsabilité Technique
engine/	Moteur de Jeu Core : Contient la boucle principale (<code>game.py</code>), la physique, et la génération procédurale des cartes (<code>map.py</code>).
engine/rl.py	Cerveau IA : Implémentation de l'agent Q-Learning, gestion de la matrice Q et des hyperparamètres (α, γ, ϵ).
entities/	Entités Vivantes : Gestion des PV, des inventaires et des états des monstres (<code>monster.py</code>) et du joueur.
database/	Persistance : Connecteurs pour le stockage relationnel (SQL - progression joueur) et NoSQL (MongoDB - logs massifs pour l'IA).
tools/	Pipeline ETL : Scripts d'administration pour l'entraînement hors-ligne (<code>train_global_model.py</code>) et l'ingestion des logs.
main.py	Point d'Entrée : Initialisation des singletons, chargement des variables d'environnement et lancement de la boucle graphique.

TAB. 2 – Description fonctionnelle de l'arborescence du projet Roguia.

A.2 Procédure de Déploiement Utilisateur

Contrairement à l'environnement de développement, la version de production ne nécessite pas l'installation de l'interpréteur Python.

1. **Téléchargement** : Se rendre sur le portail de distribution officiel pour récupérer la dernière *Release* stable :
<https://raykeshr.github.io/PFE-Roguelike/telechargements.html>
2. **Exécution** : Lancer l'exécutable `Roguia.exe`.
3. **Authentification Sécurisée** : L'accès au prototype est protégé. Lors de l'invite de connexion, saisir la phrase de passe suivante (respecter la casse) :

MotDePasseHyperSécuriséSansEspaceAvecDesMajusculesAuxDébutDeChaqueMot

A.3 Chaîne d'Industrialisation (Packaging)

Afin de garantir la reproductibilité et la distribution du projet sans contraindre l'utilisateur final à configurer un environnement Python complexe, nous avons mis en place une pipeline de *Build* automatisée :

1. **Gel de l'environnement (Freezing)** : Utilisation de `PyInstaller` pour encapsuler l'interpréteur Python et les dépendances lourdes (`numpy`, `pandas`, `pygame`) dans un exécutable autonome (Standalone).
2. **Création de l'Installeur** : Scripting `Inno Setup` (Code Pascal) pour générer un assistant d'installation standard Windows (`.exe`), gérant la création des raccourcis et l'injection sécurisée des variables d'environnement (`.env`).

Cette démarche transforme un prototype de recherche académique en un logiciel "prêt-à-jouer".



ANNEXE :

Extrait de nos table SQL :

User :

id int4	usern... var...	email varchar	mdp varchar	created_at timestamp
2	Sabri	NULL	\$2b\$12\$D0f2W3ghINbVrldG4CHzku5se\	2025-11-11 22:26:30.40647
3	RAYKESH	NULL	\$2b\$12\$kqWkLh9F.wMwiUqW/1369bOmE	2025-11-12 07:58:12.252853
4	Silver	NULL	\$2b\$12\$BswmDiWLpzR84wRLjcl8FOWk\	2025-11-12 09:21:53.583033

Player :

id int4	ut... i.	nom varchar	niveau int4	xp int4	pv int4	q_table_path varchar
2	2 →	Monster Slayer	7	500	105	models/qtables/joueur_2.pkl
3	3 →	RAYKESH	3	150	105	models/qtables/joueur_3.pkl
4	3 →	raykesh	7	275	35	models/qtables/joueur_4.pkl
5	4 →	monster hunter	2	0	85	models/qtables/joueur_5.pkl
6	5 →	Yoi	3	125	100	models/qtables/joueur_6.pkl

Items :

id integer	nom varchar(100)	description text	rarity varchar(50)	item_th varchar	damage integer	defense integer	range double precision	category varchar(50)	durability integer	potency integer	duration integer
1	Epée émoussée	Une épée émoussée mais ei	common	weapon	5	0	1	melee	90	0	0
2	Hache de guerre	Une hache lourde capable c	common	weapon	12	0	1	melee	80	0	0
3	Arc de chasseur de champi	Un arc léger et précis pour i	rare	weapon	8	0	5	distance	70	0	0
4	Gros bâton magique	Un bâton imprégné de p	epic	weapon	15	0	4	magic	60	0	0
5	Dague empoisonnée des té	Une petite lame recouverte	rare	weapon	7	0	1.5	melee	85	10	3
6	Bouclier en bois de boulot	Un bouclier simple mais util	common	weapon	0	5	0	shield	100	0	0
7	Potion de soin	Redonne progressivement c	common	potion	0	0	0	health	0	50	5
8	Potion de force	Augmente temporairement	common	potion	0	0	0	strength	0	20	10

Inventaire :

id int4	joueur_id int4	item_id int4	quantite int4
200	2 →	1 →	3
201	2 →	15 →	2
202	2 →	2 →	2
203	2 →	9 →	2
204	2 →	17 →	1
205	2 →	7 →	2

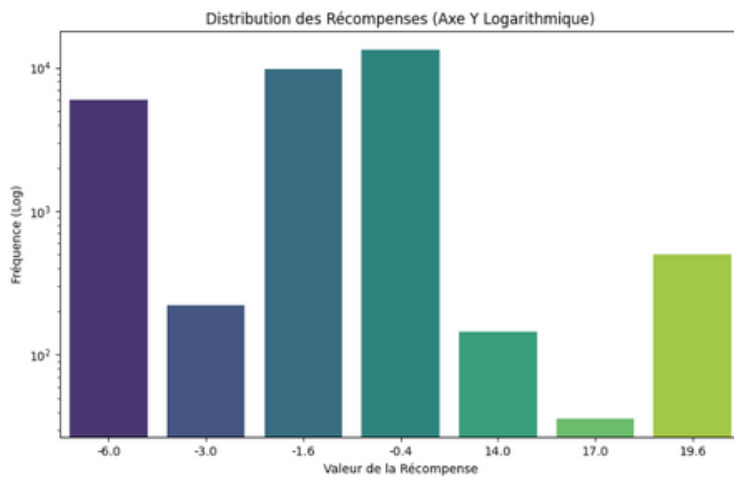


ANNEXE :

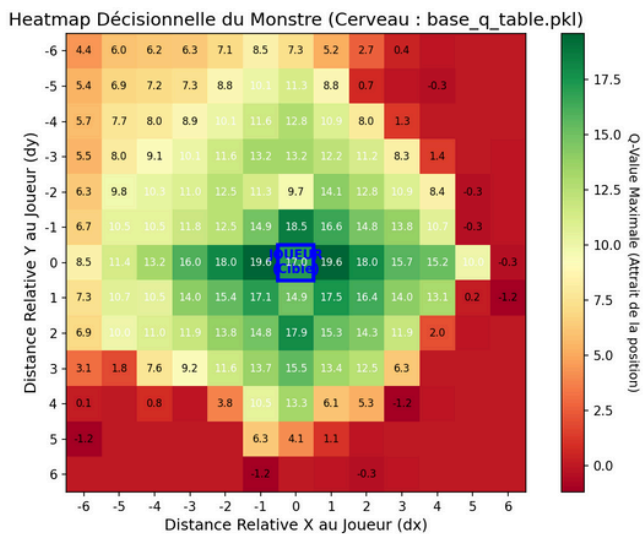
Exemple de log collecté pour l'entraînement du super monstre:

```
_id: ObjectId('690dce3384e8f9b319b301d8')
type: "transition"
ts: "2025-10-23T20:48:58.832523Z"
episode_id: "0aa6611c-71a7-43e2-8738-2ab3a243e826"
monster_id: 3046402594800
▼ s : Array (2)
  0: -6
  1: -3
  a : 0
  r : -0.4
▼ s2 : Array (2)
  0: -6
  1: -2
done: false
tick: 85
► pos : Array (2)
  location: "room"
```

Graphique de la Répartition des récompenses :



HeatMap décisionnelle d'un monstre de base :



2025 - 2026

MERCI !



*Chrisphen Ringuet - Sabri Messaoudi - Raykesh Ravirooban - Coumba Mbaye - Maxence Klein
Supervised by : Armen Petrossian & Ronan Costaouec*